Caminos de largo k

El enunciado del ejercicio dice: Tenemos un grafo representado con una matriz de adyacencia A. Dicha matriz tiene únicamente unos y ceros (según si dos vértices son adyacentes, o no). Implementar un algoritmo de **división y conquista** que, dada la matriz A y un valor k, devuelva la cantidad de caminos de longitud k que hay en el grafo correspondiente. Analizar y justificar detalladamente la complejidad del algoritmo implementado. Tener mucho cuidado al analizar la complejidad; es probable que no puedas aplicar el teorema maestro. En dicho caso, como parte del análisis de la complejidad, explicar por qué no es aplicable el teorema (nuevamente, es probable que no puedas aplicarlo).

Recomendamos recordar que:

- $A^m[i][j]$ nos dice la cantidad de caminos de longitud m que hay entre i y j.
- Que la multiplicación de matrices se puede considerar como una operación que consume $\mathcal{O}\left(n^{\log_2 7}\right)$, siendo n la dimensión de la matriz (cuadrada). Si bien este algoritmo es de división y conquista, no se pide ni es de interés que implementen nada al respecto de esto. Suponé que está disponible la función `multiplicar_matrices(A, B)`.

Análisis

En función de lo que nos piden recordar, podemos ver que lo que nos sirve calcular es A^k. Si tenemos esa matriz, automáticamente lo único que debemos hacer es recorrer la matriz sumando los valores y listo.

Entonces primero se podría pensar en un algoritmo trivial:

```
def potenciar(a, k):
    resultado = identidad(len(a))
    for i in range(k):
        resultado = multiplicar_matrices(resultado, a)
    return resultado
```

Esto va a ser un clarito $\mathcal{O}(kn^{\log_2 7})$. Claramente no estamos usando D&C, y vamos a poder mejorarlo.

Mejora y código

Algo mejor es recordar la exacta misma idea de potenciación eficiente. Solo cambia *cómo* es esa operación de potencia, que luego la analizamos.

Recordamos que para cualquier cosa, sea número o matriz: $a^m = a^{\frac{m}{2}} \cdot a^{\frac{m}{2}} = \left(a^{\frac{m}{2}}\right)^2$

Es decir, si calculamos ese valor, es simplemente multiplicarlo por si mismo y listo. Lo que hay que considerar es que, como trabajamos con enteros, hay que tener cuidado con que sea un número impar, pero eso es fácil de considerar con un simple if:

```
def potenciar(a, k):
    if k == 1:
        return a
    pot_mitad = potenciar(a, k//2)
    resultado = multiplicar_matrices(pot_mitad, pot_mitad)
    if k % 2 == 1:
        resultado = multiplicar_matrices(resultado, a)
    return resultado

def caminos(a, k):
    ak = potenciar(a, k)
    resul = 0
    for i in range(len(a)):
        for j in range(len(a)):
            resul += ak[i][j]
    return resul
```

Complejidad

Está claro que la complejidad de nuestro algoritmo va a terminar siendo:

$$\mathcal{O}\left(n^2 + potenciar\right)$$

Recorrer la matriz luego de potenciar es claro cuanto consume, nos falta ver cuánto cuesta `potenciar`. Tenemos que tener cuidado y plantear la ecuación de recurrencia. Es muy fácil caer en errores y tratar de poner primero los valores de `A`, `B` y `C` sin tener una ecuación de recurrencia, y cometer un error enorme.

Un error común sería decir: $T(n) = T\left(\frac{n}{2}\right) + \mathcal{O}\left(n^{\log_2 7}\right)$. Esto no sería correcto, porque la matriz no se va haciendo más pequeña. Siempre multiplicamos la matriz con mismo tamaño. Acá tenemos 2 variables, no una:

$$T(n,k) = T\left(n, \frac{k}{2}\right) + \mathcal{O}\left(n^{\log_2 7}\right); T(n,1) = \mathcal{O}(1)$$

Pueden sacar incluso a n de la recurrencia pero no que el costo $\mathcal{O}\left(n^{\log_2 7}\right)$ es fijo. No podemos aplicar el teorema maestro en estas condiciones.

Entonces.. ¿ya está? ¿perdimos? No, todavía hay que agarrar la pala. En este caso al menos no termina siendo tan complicado resolver esa recurrencia como podría llegar a ser en otros casos. En este caso sabemos que efectivamente vamos a tener $\log k$ invocaciones recursivas (esta cantidad **no** depende de n). Podemos incluso plantear que la cantidad de llamados recursivos pueden verse como $Cant(k) = Cant\left(\frac{k}{2}\right) + 1$. Sea más intuitivo o más analítico, es una cantidad logarítmica. Como todo lo demás que hacemos en la función recursiva es de tiempo constante, lo que importa es el `multiplicar_matrices(...)` (que se ejecute eventualmente 2 veces no nos cambia la complejidad). Es decir, se va a terminar ejecutando $\log n$ veces una función que siempre consume $\mathcal{O}\left(n^{\log_2 7}\right)$, por lo tanto, la complejidad de todo nuestro algoritmo de D&C es $T(n) = \mathcal{O}\left(n^{\log_2 7} \log k\right)$, lo cual es mejor que $\mathcal{O}\left(n^{\log_2 7} k\right)$.

Entonces en total nos va a quedar:

$$\mathcal{O}\left(n^2 + n^{\log_2 7} \log k\right) = \mathcal{O}\left(n^{\log_2 7} \log k\right)$$

(Notar que $\log_2 7 > 2$ así que por completo podemos terminar de sacar la última iteración de consideración)